

Machine Learning Approaches for Malware Detection in Android Devices

Alice Smith¹, Bob Jones²

¹*Department of Computer Science, University of Technology, London, UK*

²*Cybersecurity Research Lab, TechCorp, London, UK*

Corresponding author: alice.smith@example.com

July 7, 2026

Abstract

This paper proposes a deep learning framework for the automated detection of malicious applications in Android environments. Leveraging a custom Convolutional Neural Network (Convolutional Neural Network (CNN)) architecture, the system processes static features extracted directly from the Android Package (APK) files, translating application opcodes into a two-dimensional spatial representation. The proposed architecture significantly reduces the manual feature engineering overhead while achieving high classification accuracy. Experimental results on a dataset of 50,000 benign and malicious applications demonstrate that the model achieves an accuracy of 98.5% with a false positive rate of less than 1.2%, outperforming state-of-the-art heuristic methods.

Keywords: Machine Learning, Android, Malware Detection, Deep Learning, CNN, Security

1 Introduction

The widespread adoption of the Android Operating System (OS) has made it a prime target for malicious software. Traditional signature-based detection mechanisms struggle to keep pace with the rapid evolution of malware and zero-day threats. To address these challenges, Machine Learning (ML) techniques have emerged as a robust alternative, capable of identifying complex patterns in application behavior [1].

In this paper, we propose a static analysis approach that represents APK files as opcode sequences, which are subsequently transformed into images. This allows us to apply computer vision techniques, specifically a CNN, to classify the applications.

Deep learning architectures, such as the ones discussed in [1], have proven highly effective in various domains, ranging from image recognition to natural language processing. By leveraging these advancements, our approach bypasses the need for extensive manual feature engineering. Instead, the CNN learns optimal spatial hierarchies of features directly from the raw opcodes extracted from the APK.

This paper is structured as follows: [Section 2](#) describes the proposed feature extraction pipeline and the network architecture. [Section 3](#) provides implementation details and the PyTorch source code. [Section 4](#) presents the experimental setup and the classification performance. Finally, [Section 5](#) concludes the paper.

Algorithm 1: Opcode sequence to spatial tensor mapping

Input: Raw application package *APK*

Output: Spatial tensor representation X_S

```

1  $B \leftarrow \text{Decompile}(APK)$ ;
2  $O \leftarrow \text{ExtractOpcodes}(B)$ ;
3  $L \leftarrow \text{Length}(O)$ ;
4  $W \leftarrow \text{CalculateWidth}(L)$ ;
5  $X_S \leftarrow \text{InitializeMatrix}(W, W)$ ;
6 for  $i \leftarrow 0$  to  $L - 1$  do
7    $row \leftarrow \lfloor i/W \rfloor$ ;
8    $col \leftarrow i \bmod W$ ;
9    $X_S[row, col] \leftarrow \text{Vectorize}(O[i])$ ;
10 return  $X_S$ ;

```

2 Proposed Methodology

2.1 Feature Extraction

The first step involves extracting the compiled Dalvik bytecode from the APK. The sequence of opcodes $S = (o_1, o_2, \dots, o_N)$ is mapped into a vector space. We define the probability of an opcode sequence belonging to a malware family M as the conditional expectation:

$$P(M | S) = \mathbb{E} \Pr\{y = 1 | X_S\} \quad (1)$$

where X_S represents the tensor representation of the opcode sequence.

The extraction pipeline is formalized in [Algorithm 1](#).

2.2 Network Architecture

The core of our ML model is a deep CNN. The activation function used across the hidden layers is the Rectified Linear Unit (ReLU), defined mathematically as:

$$f(x) = \max(0, x) \quad (2)$$

During the training phase, our objective is to minimize the binary cross-entropy loss $\mathcal{L}(\theta)$. The optimal network parameters θ^* are obtained via stochastic gradient descent:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) \quad (3)$$

3 Implementation Details

The model is implemented in Python using the PyTorch framework. Listing 1 illustrates the definition of the primary convolutional module. We exploit the spatial locality of the opcodes using 3×3 convolutional kernels.

```

1 import torch
2 import torch.nn as nn
3
4 class MalwareCNN(nn.Module):
5     def __init__(self):
6         super(MalwareCNN, self).__init__()
7         # First convolutional block
8         self.conv1 = nn.Conv2d(1, 32,
9                                 kernel_size=3, padding=1)
10        self.relu = nn.ReLU()
11        self.pool = nn.MaxPool2d(2, 2)
12
13        # Fully connected classifier
14        self.fc = nn.Linear(32 * 16 * 16, 1)
15        self.sigmoid = nn.Sigmoid()
16
17    def forward(self, x):
18        x = self.pool(self.relu(self.conv1(x)))
19        x = x.view(x.size(0), -1) # Flatten tensor
20        x = self.sigmoid(self.fc(x))
21        return x

```

Listing 1: PyTorch implementation of the CNN architecture

4 Experimental Results

To evaluate the effectiveness of the proposed architecture, we compiled a diverse dataset of 50,000 Android applications, comprising an equal split of benign software and known malware families. The dataset was partitioned into 80 % for training, 10 % for validation, and 10 % for testing.

The model was trained for 50 epochs using the Adam optimizer with a learning rate of 1×10^{-3} . The training process was executed on a workstation equipped with a modern GPU to accelerate the tensor operations.

As shown in Table 1, the proposed CNN significantly outperforms traditional machine learning baselines such as Random Forests and Support Vector Machines. The spatial feature representation allows the network to capture complex, non-linear relationships between malicious instruction sequences that are often missed by simple n-gram analysis techniques.

Table 1: Performance comparison on the test dataset.

Model	Accuracy	Precision	Recall
Random Forest	89.4 %	88.1 %	90.2 %
SVM (RBF kernel)	91.2 %	90.5 %	91.8 %
Proposed CNN	98.5 %	98.2 %	98.8 %

5 Conclusion

We have presented an effective ML-based framework for Android malware detection. By treating APK opcodes as images, a CNN can automatically learn discriminative features without manual engineering. Future work will investigate the resilience of this architecture against adversarial obfuscation techniques.

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, *Deep learning*, 2015. arXiv: [1521.00561](https://arxiv.org/abs/1521.00561) [cs.LG].